

Technical Requirements Document (TRD)

Project 05 — Dual-Core RV32I SoC with Simplified Coherent Memory Subsystem

Document ID: TRD-RISCV-SOC-05

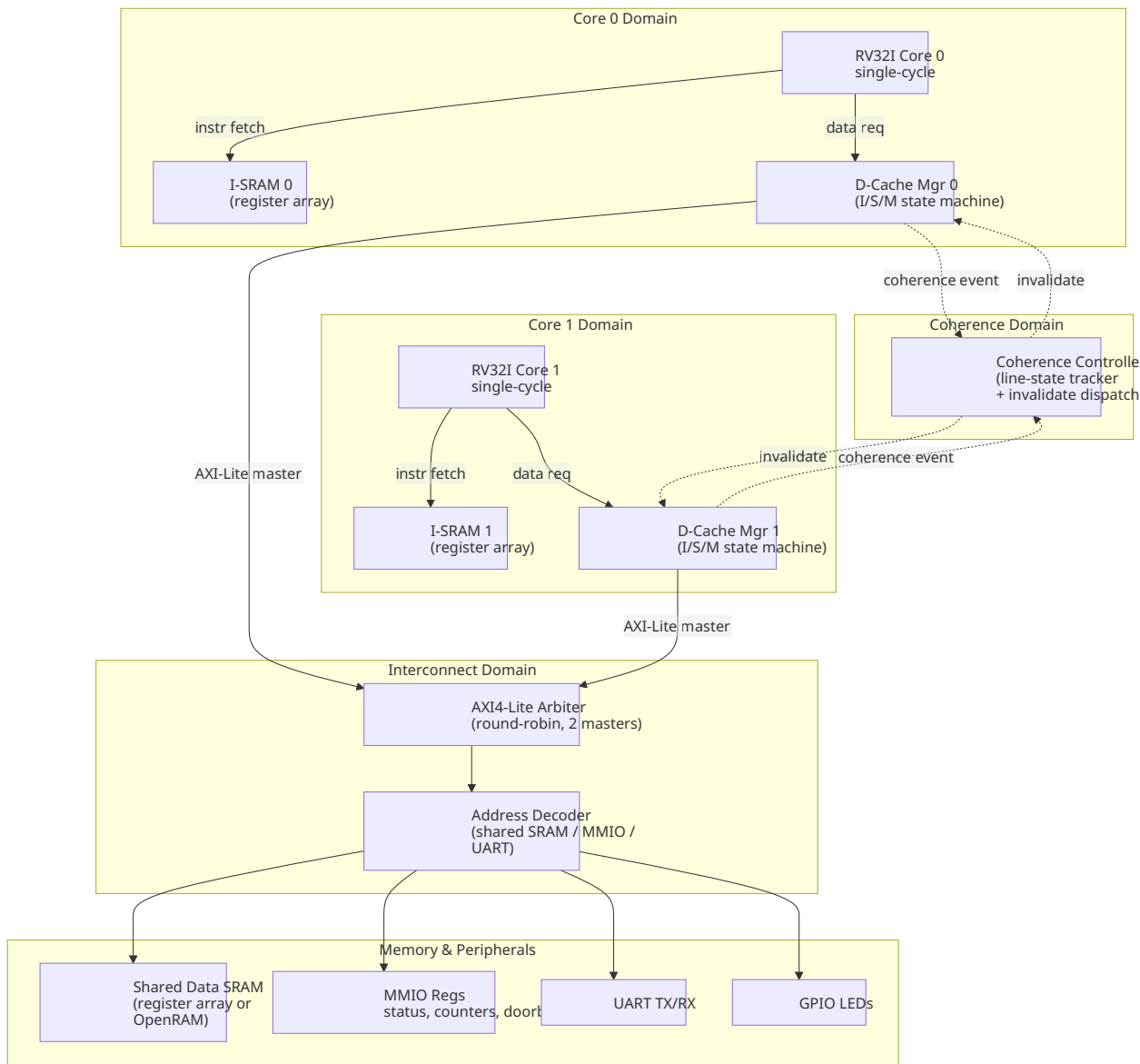
Owner: arafi

PDK: sky130A | **Flow:** OpenLane | **Bus:** AXI4-Lite | **FPGA:** Arty A7-100T / Zybo

1. Purpose

This TRD defines the **technical architecture, module specs, interfaces, protocols, memory map, signal lists, and toolchain**. It translates the PRD's functional requirements into a concrete, implementable design. It is the engineering blueprint.

2. System Architecture



2.1 Data Flow Summary

1. Core fetches instructions from private I-SRAM (never through coherence).
2. Core data accesses go through its private D-cache manager.
3. D-cache manager talks to the shared SRAM via AXI4-Lite.
4. On a write, the coherence controller is notified and invalidates the remote cache line.
5. On a read miss (or post-invalidation), the cache refetches from shared SRAM.

3. Module Specifications

3.1 RV32I Core (`rv32i_core`)

Property	Value
ISA	RV32I (base integer, 37 instructions)

Property	Value
Architecture	Single-cycle
Data width	32-bit
Register file	32 × 32-bit (x0 hardwired zero)
PC	32-bit, starts at <code>RESET_PC</code> (configurable, default 0x00000000)
Reset	Async, active-low <code>rst_n</code>

Instruction Support: All RV32I instructions: `addi, slti, sltiu, xori, ori, andi, slli, srli, srai, add, sub, sll, slt, sltu, xor, srl, sra, or, and, lb, lh, lw, lbu, lhu, sb, sh, sw, beq, bne, blt, bge, bltu, bgeu, jal, jalr, lui, auipc, ecall, ebreak`.

Interface:

Signal	Direction	Width	Description
<code>clk</code>	in	1	Clock
<code>rst_n</code>	in	1	Async active-low reset
<code>imem_addr</code>	out	32	Instruction fetch address
<code>imem_rdata</code>	in	32	Instruction word
<code>dmem_req</code>	out	1	Data access request
<code>dmem_we</code>	out	1	Write enable
<code>dmem_addr</code>	out	32	Data address
<code>dmem_wdata</code>	out	32	Write data
<code>dmem_wmask</code>	out	4	Byte-enable mask
<code>dmem_rdata</code>	in	32	Read return data
<code>dmem_ack</code>	in	1	Access complete
<code>dmem_err</code>	in	1	Access error

Implementation Note: Single-cycle means every instruction completes in 1 clock. The data interface is a simple request/ack handshake (not AXI directly — the D-cache manager translates to AXI-Lite).

3.2 Instruction SRAM (`i_sram`)

Property	Value
Type	Register array (recommended) or OpenRAM macro
Size	Configurable, e.g. 1 KB (256 words) per core
Ports	1 read port (instruction fetch)
Init	<code>\$readmemh</code> in sim; FPGA: BRAM init from <code>.mem</code> file

Interface: Simple synchronous read; address in, instruction out next cycle (or combinational for register-array variant).

3.3 Private Data Cache (`d_cache`)

Property	Value
Organization	Direct-mapped
Lines	2–4 (configurable; start with 4)
Line size	1 word (32-bit)
Fields per line	<code>valid</code> , <code>tag</code> , <code>data[31:0]</code> , <code>state[1:0]</code>
Write policy	Write-through, write-allocate
Coherence states	I (2'b00), S (2'b01), M (2'b10)

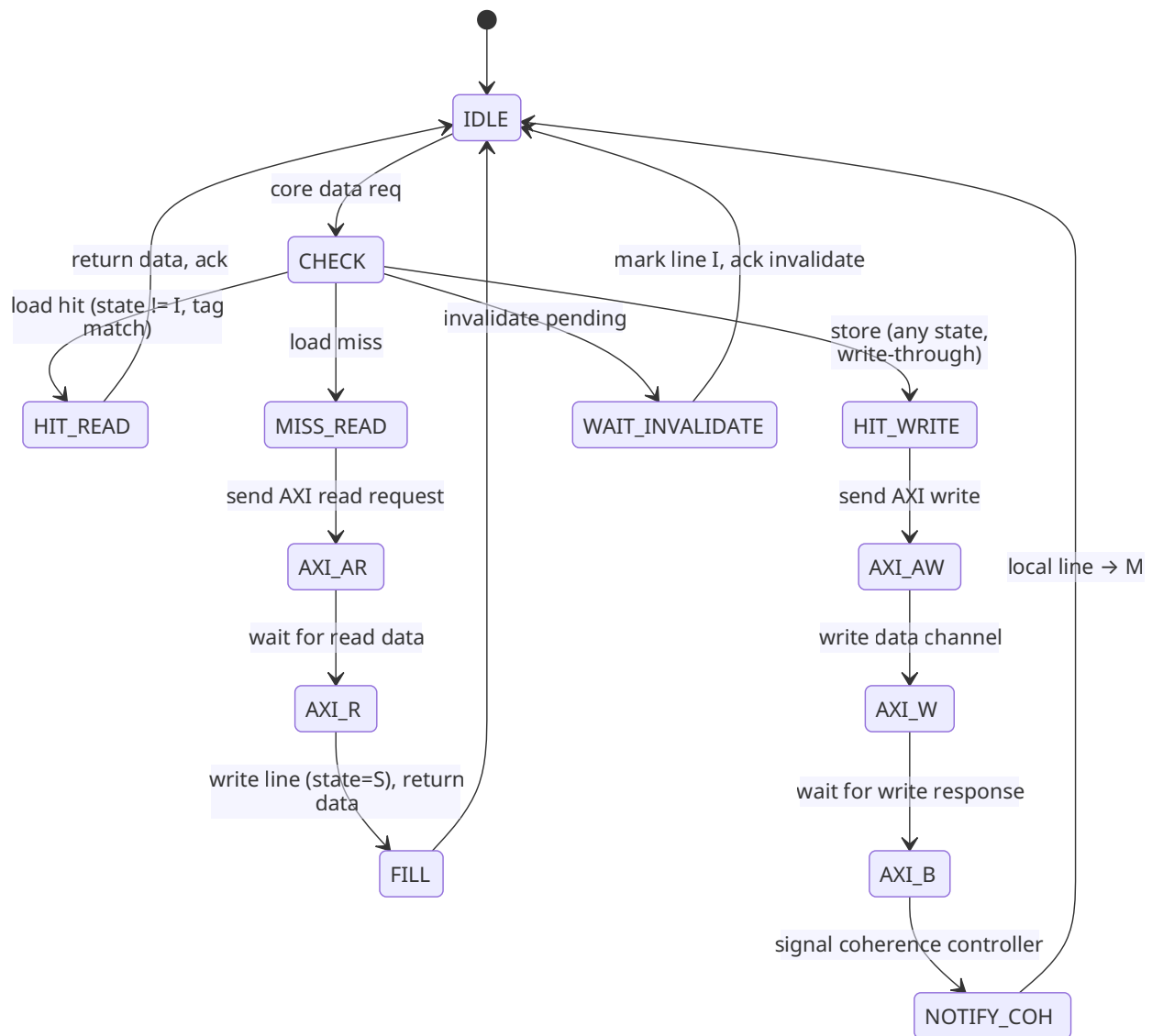
State Encoding:

State	Code	Meaning
I	2'b00	Invalid — line not present or stale
S	2'b01	Shared — clean copy from shared SRAM
M	2'b10	Modified — this core wrote through; owns latest value

Cache Line Structure:

```
typedef struct packed {  
    logic valid;  
    logic [W-1:0] tag;  
    logic [31:0] data;  
    logic [1:0] coh_state; // I/S/M  
} cache_line_t;
```

Cache Manager FSM (`d_cache_mgr`):



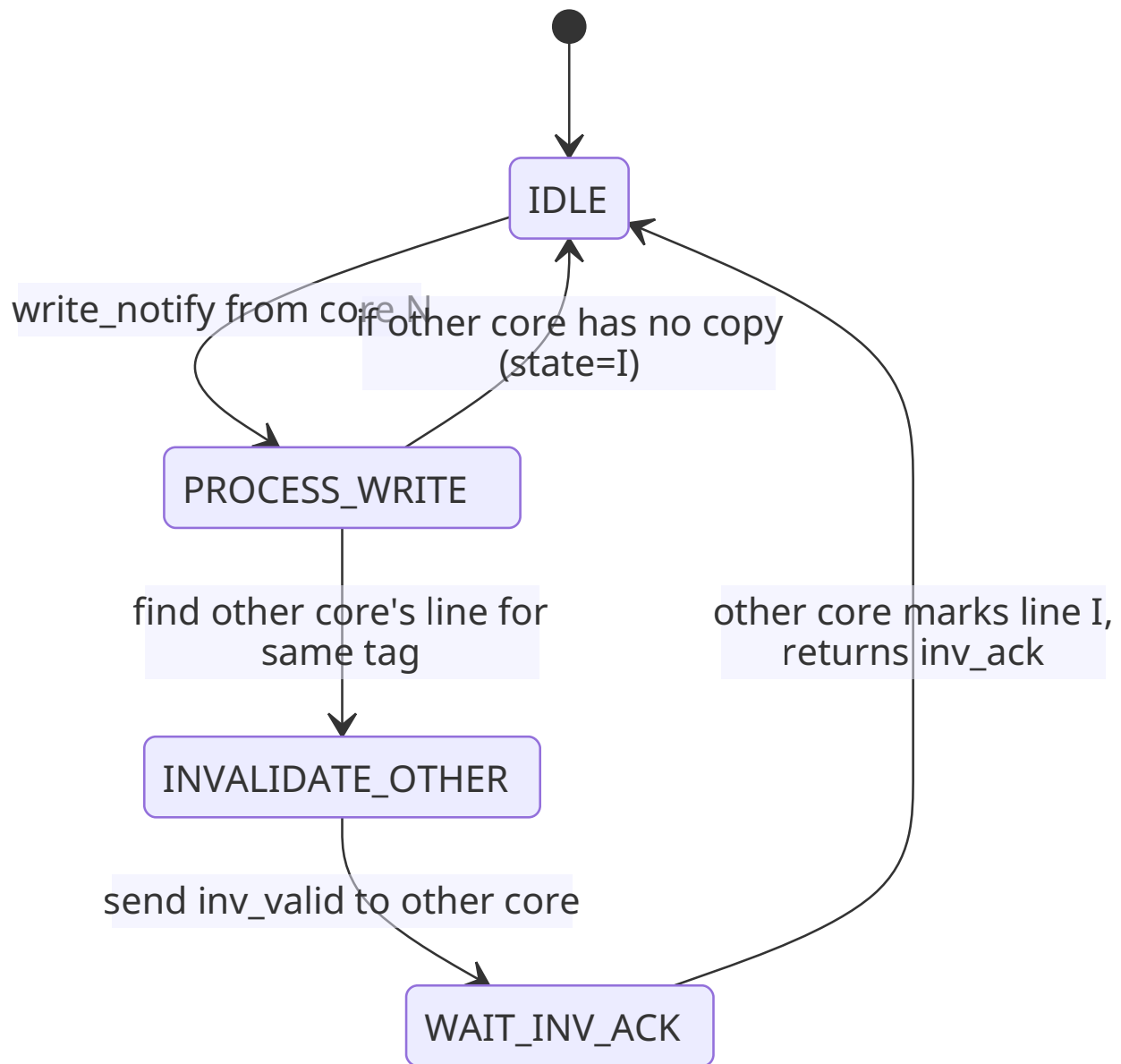
Invalidation Interface (from coherence controller):

Signal	Direction	Description
<code>inv_valid</code>	in	Invalidation request for this cache
<code>inv_addr</code>	in	Address (or line index) to invalidate
<code>inv_ack</code>	out	Invalidation complete

3.4 Coherence Controller (`coherence_ctrl`)

Property	Value
Role	Track per-line, per-core coherence state; dispatch invalidations
Line tracking	Mirrors the same line set as the caches (4 lines × 2 cores = 8 state entries)
Trigger	Any write-through from either core's cache manager

Coherence Controller FSM:



Per-Line State Table (coherence controller's view):

Line	Core 0 state	Core 1 state	Action on Core 0 write
L0	S	S	Core1 L0 → I
L0	M	I	No invalidation needed
L0	I	S	Core1 L0 → I
L0	I	I	No invalidation needed

Interface:

Signal	Direction	Description
<code>write_notify</code>	in (from cache mgr)	Core N completed a write to address A
<code>write_core_id</code>	in	Which core wrote (0 or 1)
<code>write_addr</code>	in	Address that was written
<code>inv_valid</code>	out (to other cache)	Invalidate this line

Signal	Direction	Description
<code>inv_addr</code>	out	Line to invalidate
<code>inv_ack</code>	in	Other cache acknowledges invalidation
<code>coh_status</code>	out	Current coherence states (for MMIO debug)

3.5 AXI4-Lite Arbiter (`axi_lite_arbiter`)

Property	Value
Masters	2 (one per core's D-cache manager)
Slaves	1 (via address decoder)
Arbitration	Round-robin (or fixed priority)
Deadlock-free	Yes — each transaction has bounded completion

Arbiter Logic: At any idle point, grant the master that has been waiting longest. Once a master is granted, it holds the bus until its transaction completes (AXI4-Lite: one AW+W+B, one AR+R).

3.6 AXI4-Lite Address Decoder (`axi_lite_decoder`)

Address Range	Target	Description
<code>0x0000_0000 - 0x0000_0FFF</code>	Shared Data SRAM	4 KB shared data
<code>0x0001_0000 - 0x0001_00FF</code>	MMIO regs	Status, counters, doorbell
<code>0x0001_0100 - 0x0001_01FF</code>	UART	TX/RX/data regs
<code>0x0001_0200 - 0x0001_02FF</code>	GPIO/LEDs	LED output reg
Other	DECERR	Unmapped → error response

AXI4-Lite Slave Port (per target):

Channel	Signals
AW	<code>awvalid</code> , <code>awaddr</code> , <code>awready</code> , <code>awprot</code>
W	<code>wvalid</code> , <code>wdata</code> , <code>wstrb</code> , <code>wready</code>
B	<code>bvalid</code> , <code>bresp</code> , <code>bready</code>
AR	<code>arvalid</code> , <code>araddr</code> , <code>arready</code> , <code>arprot</code>
R	<code>rvalid</code> , <code>rdata</code> , <code>rresp</code> , <code>rready</code>

Response Codes: `OKAY` (2'b00), `SLVERR` (2'b10), `DECERR` (2'b11).

3.7 Shared Data SRAM (`shared_sram`)

Property	Value
Size	4 KB (1024 words)

Property	Value
Type	Register array (recommended) or OpenRAM macro
Ports	1 RW port (AXI-Lite slave)
Latency	1 cycle (register array)

3.8 MMIO Registers (`mmio_regs`)

Offset	Name	Access	Description
0x00	<code>COH_STATUS</code>	RO	Current coherence states for all lines (both cores)
0x04	<code>INV_COUNT</code>	RO	Total invalidation events counter
0x08	<code>HIT_COUNT</code>	RO	Total cache hit counter
0x0C	<code>MISS_COUNT</code>	RO	Total cache miss counter
0x10	<code>DOORBELL</code>	RW	Core-to-core doorbell (write to signal other core)
0x14	<code>CONTROL</code>	RW	Control bits (reset counters, enable/disable coherence)

3.9 UART (`uart_core`)

Property	Value
Baud	115200 (configurable)
Data	8N1
TX	MMIO-writable TX data register
RX	MMIO-readable RX data register (with status)
Interface	AXI4-Lite slave

UART Register Map (within UART address space):

Offset	Name	Access	Description
0x00	<code>TX_DATA</code>	W	Write byte to transmit
0x04	<code>TX_STATUS</code>	R	TX ready (bit 0)
0x08	<code>RX_DATA</code>	R	Received byte
0x0C	<code>RX_STATUS</code>	R	RX valid (bit 0)

3.10 GPIO/LED (`gpio_led`)

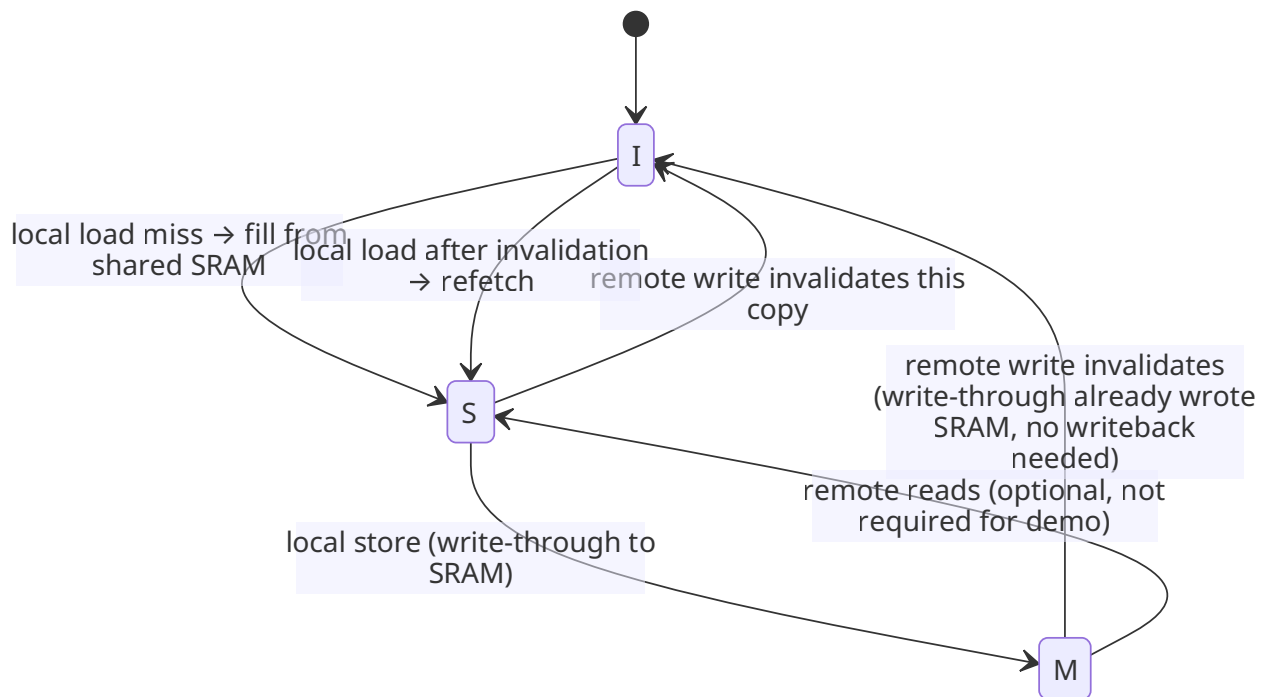
Property	Value
Width	4–8 bits
Interface	AXI4-Lite slave (single write reg)
Output	<code>led[7:0]</code> pins

LED Assignment:

LED	Meaning
LED0	Core 0 active (heartbeat)
LED1	Core 1 active (heartbeat)
LED2	Coherence event pulse (invalidation fired)
LED3	Cache hit (blinks on hit)
LED4	Cache miss (blinks on miss)
LED5	Error (lit on bus error)
LED6-7	Spare / demo status

4. Coherence Protocol Specification

4.1 State Transitions



4.2 Transaction Scenarios

Scenario A: Core 0 writes, Core 1 reads (the core demo)

- Core 0: SW r1, 0(shared_addr) → D-cache 0 write-through
- D-cache 0 → AXI write to shared SRAM
- Coherence controller: Core 0 line → M, notify invalidation of Core 1 line
- Core 1 D-cache: invalidate line → I
- Core 1: LW r2, 0(shared_addr) → miss (state=I) → fetch from SRAM → gets new value

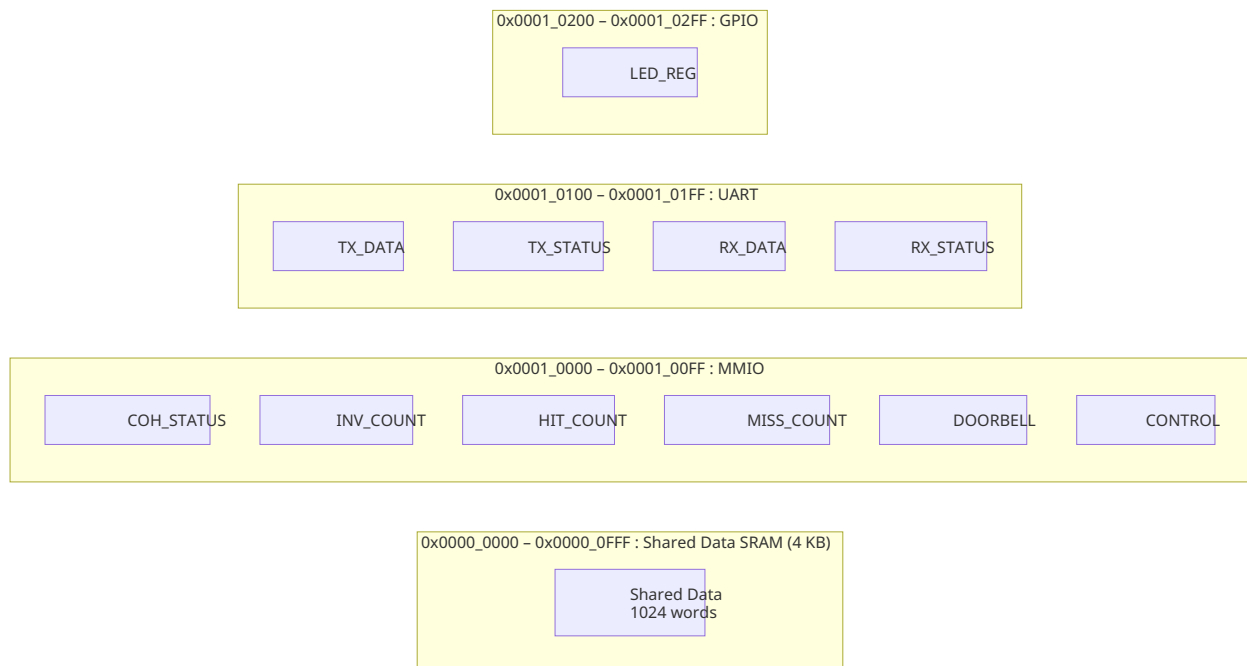
Scenario B: Simultaneous writes to same address

1. Both cores issue SW to same addr in same cycle
2. Arbiter picks winner (e.g., Core 0 by round-robin priority)
3. Core 0 writes → Core 1 line invalidated
4. Core 1 writes → Core 0 line invalidated
5. Final SRAM value = last writer (Core 1, second in arbiter order)
6. Both caches end in I state for that line

Scenario C: No cached copy in other core

1. Core 0 writes to addr A
2. Coherence controller checks: Core 1 has no valid copy of A (state=I)
3. No invalidation sent – Core 0 line → M

5. Memory Map



Note: I-SRAMs are private per core and are NOT memory-mapped in the shared space. They have their own private fetch path outside AXI.

6. Interface Contracts

6.1 Core ↔ D-Cache Manager (internal, non-AXI)

Simple request/ack handshake:

Signal	Dir	Description
<code>req</code>	core→cache	Data access request
<code>we</code>	core→cache	1=write, 0=read
<code>addr[31:0]</code>	core→cache	Data address
<code>wdata[31:0]</code>	core→cache	Write data
<code>wmask[3:0]</code>	core→cache	Byte enables
<code>rdata[31:0]</code>	cache→core	Read return data
<code>ack</code>	cache→core	Transaction complete
<code>err</code>	cache→core	Error (bus fault)

6.2 D-Cache Manager ↔ AXI-Lite (master port)

Standard AXI4-Lite master signals (5 channels). The D-cache manager implements the AXI-Lite master FSM.

6.3 Coherence Controller ↔ D-Cache Managers (sideband)

Signal	Dir	Description
<code>write_notify</code>	cache→coh	A write completed on this core
<code>write_addr</code>	cache→coh	Address written
<code>inv_req</code>	coh→cache	Invalidate this line
<code>inv_line_idx</code>	coh→cache	Which line to invalidate
<code>inv_ack</code>	cache→coh	Invalidation done

7. Clocking & Reset

Domain	Clock	Source
SoC (all cores, caches, coherence, bus, memory)	<code>core_clk</code>	Single clock, synchronous design
FPGA	<code>core_clk</code> from MMCM	Board ref clock (100/125 MHz) → MMCM → 50 MHz
UART	<code>core_clk</code>	Baud divider from <code>core_clk</code>

Reset:

- Single async active-low `rst_n`.
- Reset synchronizer at top level (2-flop synchronizer) to release reset synchronously to `core_clk`.
- On reset: all caches → I, all counters → 0, all state regs → known, PC → `RESET_PC`.

8. Timing Constraints (SDC)

```
# Core clock – start conservative for sky130
```

```

create_clock -name core_clk -period 50.000 [get_ports clk]
# 50 ns = 20 MHz target (tighten to 25 ns / 40 MHz if closes)
# Input/output delays (conservative, refine after placement)
set_input_delay -max 10.0 -clock core_clk [all_inputs]
set_output_delay -max 10.0 -clock core_clk [all_outputs]
# False path / multicycle if needed (e.g., UART baud counter)
# set_false_path -from [get_pins uart_baud_reg/Q] -to [get_pins uart_tx_reg/D]
# Asynchronous reset
set_false_path -from [get_ports rst_n]

```

FPGA SDC (Xilinx Vivado):

```

create_clock -name sys_clk -period 10.0 [get_ports clk_in]
# 100 MHz Arty ref → MMCM → 50 MHz core_clk (20 ns)
create_generated_clock -name core_clk [get_pins mmcm/clk_out0]

```

9. Toolchain

Stage	Tool	Notes
RTL simulation	Icarus Verilog / Verilator / Vivado XSim	Directed TB + UVM (UVM needs a UVM-capable simulator — use <code>verilator --UVM</code> or a commercial sim if available, or <code>xrun</code> / <code>vsim</code>).
ASIC synthesis	Yosys (via OpenLane)	<code>sky130_fd_sc_hd</code> standard cells.
ASIC physical design	OpenROAD (via OpenLane)	Floorplan → PDN → placement → CTS → routing.
DRC / LVS	Magic + Netgen (via OpenLane) / KLayout	OpenLane runs these; KLayout for visual LVS.
FPGA synthesis	Xilinx Vivado	Arty A7-100T: xc7a100t; Zybo: xc7z010/xc7z020 (PL-only).
FPGA simulation	Vivado XSim / Verilator	Behavioral sim before board.
RISC-V toolchain	RISC-V GCC (<code>riscv32-unknown-elf-gcc</code>)	Compile C/asm → hex for I-SRAM init.

10. Directory Structure

```

riscv_soc_coh/
├─ rtl/
│ └─ core/
│   │ └─ rv32i_core.sv
│   │ └─ reg_file.sv
│   │ └─ alu.sv
│   └─ control_unit.sv

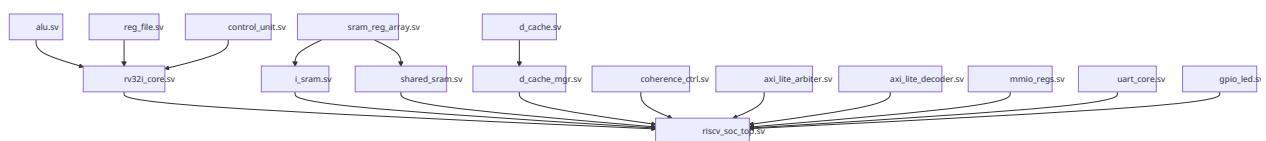
```

```

├── memory/
│   ├── i_sram.sv
│   ├── shared_sram.sv
│   └── sram_reg_array.sv
├── cache/
│   ├── d_cache.sv
│   └── d_cache_mgr.sv
├── coherence/
│   └── coherence_ctrl.sv
├── bus/
│   ├── axi_lite_arbiter.sv
│   ├── axi_lite_decoder.sv
│   └── axi_lite_if.sv
├── peripheral/
│   ├── mmio_regs.sv
│   ├── uart_core.sv
│   └── gpio_led.sv
├── top/
│   └── riscv_soc_top.sv
├── tb/
│   ├── directed/
│   │   └── tb_directed.sv
│   ├── uvm/
│   ├── env/
│   ├── agents/
│   ├── sequences/
│   ├── scoreboard/
│   └── tb_uvm.sv
├── sw/
│   ├── core0_prog.c / .s
│   ├── core1_prog.c / .s
│   └── linker.ld
├── constraints/
│   ├── soc.sdc
│   └── fpga.xdc
├── openlane/
│   └── config.json
├── fpga/
│   └── vivado/
├── docs/
│   └── architecture.md
└── Makefile

```

11. Module Dependency Graph (build order)



Build order: ALU → reg_file → control_unit → rv32i_core → i_sram → sram_reg_array → shared_sram → d_cache → d_cache_mgr → coherence_ctrl → axi_lite_arbiter → axi_lite_decoder → mmio_regs → uart_core → gpio_led → **riscv_soc_top**.

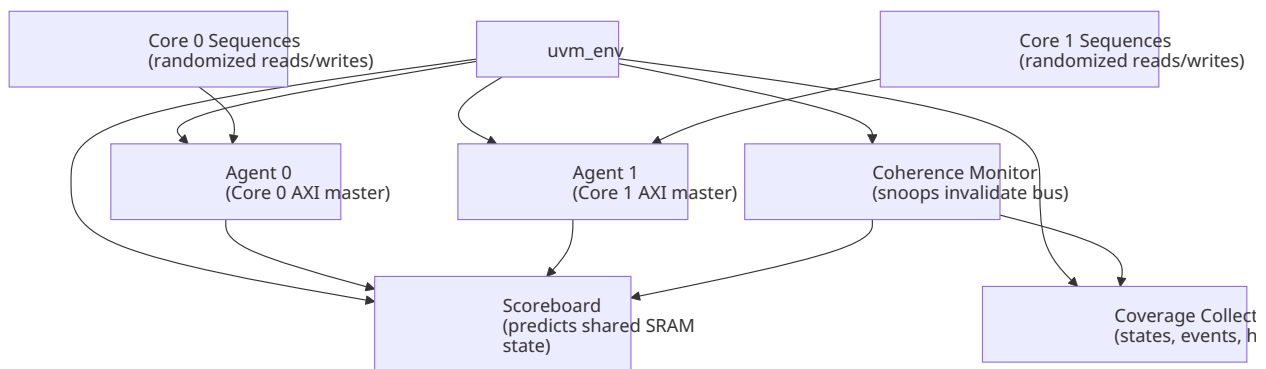
12. Verification Architecture

12.1 Directed TB

Self-checking testbench in SystemVerilog:

- Instantiates `riscv_soc_top`.
- Preloads I-SRAMs with test programs (`$readmemh`).
- Monitors UART output, MMIO counters, and shared SRAM.
- Self-checks: asserts expected values after each coherence scenario.
- Test cases: reset, normal read/write, cache hit, cache miss, cross-core invalidation, simultaneous write, error/unmapped access.

12.2 UVM Environment



UVM Components:

- **Agents** (×2): AXI4-Lite master agent per core — driver, monitor, sequencer.
- **Sequences**: constrained-random read/write sequences to shared SRAM addresses.
- **Scoreboard**: predicts expected shared SRAM value based on transaction order; compares with actual.
- **Coherence Monitor**: independently observes invalidation events; verifies invalidations fire on every cross-core write.
- **Coverage**: functional coverage for coherence states (I/S/M per core per line), hit/miss, invalidation count, simultaneous-write winner, error responses.

13. Physical Design Targets (sky130A / OpenLane)

Parameter	Target
Clock	20 MHz minimum (50 ns)
Core utilization	50–60%
Die area	As small as design allows (let OpenLane auto-size or set to ~500×500 μm)
Metal layers	sky130: up to 5/6 metal layers

Parameter	Target
DRC	0 violations
LVS	Clean (netlist match)
Congestion	No overflow (>100% routing) on any layer

14. FPGA Targets

Parameter	Arty A7-100T	Zybo (PL-only)
Device	xc7a100t-1csg324	xc7z010-1clg400 / xc7z020-1clg400
Core clock	50 MHz (MMCM from 100 MHz)	50 MHz (MMCM from 125/100 MHz)
I-SRAM	BRAM (initialized from <code>.mem</code>)	BRAM
Shared SRAM	BRAM or distributed RAM	BRAM or distributed RAM
UART	Onboard USB-UART (UART pins)	Onboard USB-UART
LEDs	4 onboard + Pmod if needed	4 onboard
Constraints	<code>.xdc</code>	<code>.xdc</code> (PL-only, no PS)